

Apuntes Semana 13

Apuntes del 21 de mayo de 2026

Steven Sequeira Araya - 2022080836

Inteligencia Artificial

stsequeira@estudiantec.cr

Resumen—Este documento resume los contenidos clave de la sesión de clase enfocados en el funcionamiento de los Modelos de Lenguaje Grandes (LLMs). Se repasan de forma sencilla los procesos de tokenización, el mapeo conceptual en espacios vectoriales y las métricas matemáticas para calcular la similitud semántica. Finalmente, se detalla el flujo de la Arquitectura RAG (Generación Aumentada por Recuperación) como solución de ingeniería ante las alucinaciones y el conocimiento estático de los modelos actuales.

Index Terms—Modelos de Lenguaje Grandes (LLM), Tokenización, Espacio Vectorial, Similitud del Coseno, Generación Aumentada por Recuperación (RAG).

I. DUDAS

I-A. Consultas sobre la Tarea #3

Durante el espacio de aclaración de dudas para la Tarea #3, se resolvieron dos aspectos fundamentales de implementación:

- **Arquitectura del Autoencoder:** Se validó como correcta la adaptación en la cual los estudiantes mantuvieron el espacio latente original, a pesar de que el modelo de referencia utilizaba una arquitectura basada en un perceptrón tradicional.
- **Limitaciones de Hydra en Jupyter Notebooks:** Se reportó que los decoradores de la librería *Hydra* presentan fallas de compatibilidad dentro de entornos interactivos de Jupyter. Como solución óptima y preparación hacia el Proyecto #2, el profe recomendó modularizar todos los componentes y *scripts* de PyTorch directamente en archivos con extensión `.py` (donde los decoradores operan correctamente) para posteriormente invocar y registrar la ejecución desde el *notebook*.

II. NOTICIAS DE LA SEMANA

II-1. Google I/O 2026: El evento central estuvo marcado por un cambio de paradigma expuesto por Sundar Pichai: la transición global desde los asistentes convencionales hacia una **era de agentes de IA** capaces de ejecutar trabajo autónomo por el usuario. Los anuncios principales incluyeron:

- **Gemini 3.5 y 3.5 Flash:** Nuevas iteraciones enfocadas críticamente en la optimización de tiempos de respuesta y velocidad de ejecución de tareas complejas.
- **Gemini Omni y AI Search:** Introducción de capacidades multimodales nativas para la creación y edición simultánea de texto, imagen y video, junto con una evolución conversacional profunda en el motor de búsqueda.

- **Gemini Spark y Ecosistema:** Lanzamiento de un asistente orientado a la organización y automatización de acciones, logrando una integración vertical más profunda en Android, Chrome, YouTube y Google Workspace.
- **Hardware y Realidad Extendida:** Impulso estratégico junto a socios comerciales para el desarrollo de Android XR y gafas inteligentes de próxima generación.

II-2. Incidentes de Ciberseguridad y Plataformas:

- **Claude (Anthropic):** Se discutió el reporte crítico de un incidente en el cual la plataforma eliminó de forma masiva y sin el consentimiento explícito la totalidad de los datos estructurados de una organización.
- **Vulnerabilidad en GitHub:** Se detectó una campaña ilícita a gran escala en la que atacantes lograron comprometer miles de repositorios privados. El vector de ataque consistió en la explotación de una extensión maliciosa que escaneó y extrajo información sensible de los códigos fuente.

III. FUNDAMENTOS DE LOS MODELOS DE LENGUAJE (LLM)

Los Modelos de Lenguaje Grandes (LLM) son la base de la inteligencia artificial moderna. Se usan tanto porque son muy versátiles: no solo entienden y generan texto, sino que también pueden trabajar con código de programación, imágenes y audio. Su gran ventaja es que organizan la información de una forma muy parecida a como lo hacemos los humanos, lo que permite automatizar tareas difíciles y resolver problemas en muchas industrias.

III-A. ¿Cómo funciona un modelo de IA por dentro?

Para entender cómo procesan la información estos modelos, hay que ver cómo es una red neuronal desde adentro. El camino que siguen los datos y cómo se transforman se puede resumir en los cuatro elementos básicos que se muestran en el Cuadro I.

Como se ve en la Fig. 1, estos componentes se conectan uno tras otro en diferentes capas, logrando que la información viaje hacia adelante hasta dar la respuesta final.

IV. EL PROCESO DE TOKENIZACIÓN

Para que un modelo de IA pueda procesar texto, primero debe traducir cada palabra, signo o símbolo a números que la computadora sí pueda entender. Este paso se conoce como **tokenización**.

Tabla I
COMPONENTES BÁSICOS DE UNA RED NEURONAL

Componente	Explicación Simple
Inputs (Entradas)	Los datos iniciales que recibe el modelo (ej. velocidad, tipo de terreno, experiencia).
Weights (Pesos)	Números que definen qué tan importante es cada dato de entrada para el resultado final.
Hidden Nodes	Las neuronas ocultas que suman las entradas, aplican un filtro matemático y descubren patrones.
Output (Salida)	El resultado final o la predicción que hace la red (ej. si va a chocar o no).

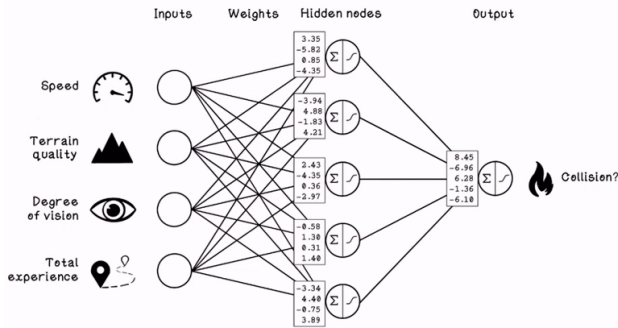


Figura 1. Esquema de una red neuronal y cómo se conectan sus partes.

IV-A. ¿Qué es la Tokenización?

La tokenización es una técnica que divide el texto original en unidades mínimas llamadas *tokens* (que pueden ser palabras completas, pedazos de palabras o letras individuales). A cada uno de estos tokens se le asigna un número de identificación único dentro del vocabulario del modelo.

Por ejemplo, una frase como "Los LLM aprenden patrones" se convierte en una lista de números como [105, 2124, 893, 5749]. En este proceso, el modelo suele pasar todo a minúsculas para estandarizar las palabras y facilitar el mapeo matemático (ver el Cuadro II).

Tabla II
EJEMPLO SIMPLIFICADO DE TOKENIZACIÓN

Token (Palabra)	ID Numérico
los	105
llm	2124
aprenden	893
patrones	5749

IV-B. Tipos de Tokenización

Dependiendo de cómo el modelo corte el texto, existen diferentes estrategias. Las más comunes y sus ventajas se resumen en el Cuadro III.

IV-C. El Reto del Significado: El Espacio Vectorial

Convertir las palabras en números de identificación (IDs) tiene un gran problema: esos números son asignados al azar y no significan nada por sí solos. El modelo no puede saber

Tabla III
TIPOS COMUNES DE TOKENIZACIÓN Y SUS VENTAJAS

Enfoque	Ventaja
Palabra	Muy simple de aplicar, pero genera listas de vocabulario gigantescas.
Carácter	Corta letra por letra. Evita el problema de encontrarse con palabras desconocidas.
Subpalabra	Corta las palabras en raíces (ej. aprend + iendo). Es el estándar actual porque ahorra espacio y entiende el contexto.
Byte-level	Trabaja con código UTF-8. Soporta cualquier tipo de símbolo, idioma o emoji.
Espacio en blanco	Separa los textos basándose solo en los espacios. Es la opción más rápida para tareas básicas.

que "perro" y "cachorro" están relacionados solo viendo sus IDs numéricos.

Para resolver esto y hacer que la IA entienda el significado real y el contexto de las palabras, los modelos avanzados organizan los tokens conectándolos de dos formas:

- **Relaciones Semánticas (Concepto):** Relacionan palabras que comparten una lógica en el mundo real, como países con sus capitales (*Grecia - Atenas*) o roles de género (*hermano - hermana*).
- **Relaciones Sintácticas (Gramática):** Relacionan palabras por cómo se escriben o su regla gramatical, como pasar un verbo al pasado (*caminar - caminé*) o convertir una palabra a plural.

IV-D. Mapeo en un Plano de Dos Dimensiones (2D)

Para que la IA pueda calcular estas relaciones, coloca cada token dentro de un mapa matemático llamado **espacio vectorial**. En este mapa, cada palabra recibe coordenadas basadas en sus características (dimensiones).

En un ejemplo simple de dos dimensiones (como se ve en la Fig. 2), podemos definir dos ejes conceptuales:

1. **Eje de Género (Vertical):** Donde el valor 1 es femenino y -1 es masculino.
2. **Eje de Realeza (Horizontal):** Donde 1 significa que pertenece a la monarquía y 0 es un ciudadano común.

Gracias a estas coordenadas, las palabras con significados parecidos quedan agrupadas de forma automática en el mapa: "mujer" recibe las coordenadas (0, 1) y "reina" las coordenadas (1, 1), compartiendo cercanía en el eje de género. Al estar en un plano matemático, la IA puede usar fórmulas simples para restar y sumar conceptos, como por ejemplo:

$$\text{Rey} - \text{Hombre} + \text{Mujer} = \text{Reina} \quad (1)$$

IV-E. Espacio Tridimensional (3D) y Alta Dimensionalidad

Si llevamos el mapa matemático un paso más allá, los modelos de IA expanden este plano hacia un **espacio en 3D** (y en la práctica, a cientos de dimensiones) para entender relaciones mucho más detalladas:

- **Más características conceptuales:** Al añadir un tercer eje como la **Edad**, junto a los de **Género** y **Realeza**, el

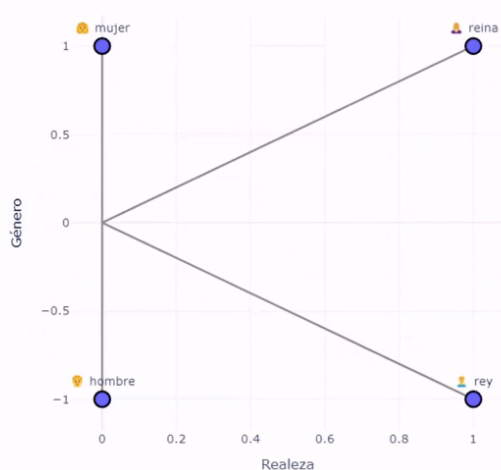


Figura 2. Distribución de palabras en un plano matemático de dos dimensiones según sus características.

modelo puede notar diferencias generacionales dentro de una misma idea.

- **Grupos ordenados en 3D:** Esto permite separar palabras con gran precisión. Por ejemplo, “reina” y “princesa” comparten las mismas características en género y realeza, pero se separan en el eje vertical por su edad. Lo mismo pasa con “hombre/niño” o “rey/príncipe”.
- **Modelos Reales:** En los LLMs reales, este espacio no tiene solo 2 o 3 dimensiones, sino cientos o miles (alta dimensionalidad) para mapear todo el lenguaje humano.

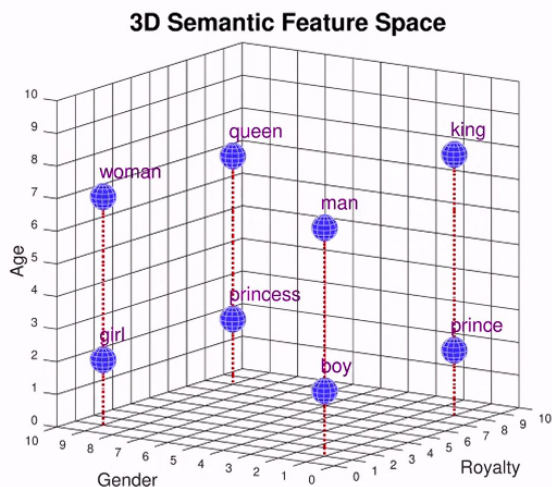


Figura 3. Representación conceptual de tokens distribuidos en un espacio de características de tres dimensiones (3D).

IV-F. Similitud entre Vectores

Al tener las palabras representadas como coordenadas en un mapa, el sistema puede usar matemática para calcular qué tan parecidas son. La regla es simple: **vectores cercanos significan palabras con significados parecidos** (como “casa” y

“hogar”), mientras que **vectores lejanos significan conceptos diferentes**.

Para medir numéricamente esta cercanía en el mapa, se usan principalmente dos fórmulas matemáticas descritas en el Cuadro IV.

Tabla IV
MÉTRICAS PARA MEDIR SIMILARIDAD ENTRE VECTORES

Métrica	¿Cómo funciona de forma simple?
<i>Distancia Euclidiana</i>	Mide la separación en línea recta entre dos puntos. Calcula qué tan lejos está un término del otro.
<i>Similitud del Coseno</i>	Evalúa el ángulo que se forma entre los dos vectores. Es la más usada porque prioriza la dirección (el significado) sobre el largo del vector.

Las fórmulas matemáticas exactas que utiliza el sistema para estos cálculos son las siguientes:

$$\text{Distancia Euclidiana: } d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2} \quad (2)$$

$$\text{Similitud del Coseno: } \text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} \quad (3)$$

Cuando el ángulo en la similitud del coseno es más pequeño, significa que los dos términos comparten un contexto casi idéntico en el modelo de lenguaje.

V. EMBEDDINGS Y CAPACIDADES DE LOS LLMs

V-A. ¿Qué son los Embeddings?

Los **embeddings** son listas de números (vectores) que representan el significado real de una palabra, un pedazo de palabra o una frase completa dentro de un mapa de alta dimensión.

A diferencia de los IDs de tokenización simples, un embedding captura el significado profundo y el contexto de las palabras. El flujo para crearlos y usarlos sigue tres pasos básicos:

1. **Tokenización:** El texto se corta en unidades mínimas (tokens).
2. **Mapeo Vectorial:** Cada token se convierte en su vector continuo (embedding) dentro del mapa matemático.
3. **Operación y Comparación:** Los vectores se operan matemáticamente (sumas o restas) y se mide su similitud para entender cómo se relacionan las ideas.

V-A1. Embeddings de Frases (Sentence Embeddings):

Cuando los modelos analizan oraciones completas en lugar de palabras sueltas, se generan embeddings de frases. Su gran ventaja es que capturan la idea global y la intención completa de un texto, permitiendo comparar significados e identificar intenciones similares aunque se usen palabras totalmente distintas.

V-B. Capacidades Emergentes

Gracias a que son entrenados con bloques gigantescos de datos y usan la arquitectura de **Transformers**, los LLMs desarrollan **capacidades emergentes**. Estas son habilidades que aparecen solas por el propio aprendizaje del modelo y que nadie programó a mano. En el Cuadro V se detallan las principales:

Tabla V
CAPACIDADES EMERGENTES DE LOS MODELOS DE LENGUAJE

Capacidad	¿En qué consiste de forma simple?
Comprensión Contextual	Entienden el significado exacto de una palabra según las oraciones que la rodean.
Generación Coherente	Redactan, traducen o resumen textos manteniendo un mismo estilo y consistencia fluida.
Razonamiento	Resuelven problemas de lógica, explican pasos detallados y trazan estrategias sencillas.
Aprendizaje en el Prompt	Adaptan su tono o comportamiento usando los ejemplos que el usuario les da en el mismo chat (<i>In-context Learning</i>).
Multitarea	Hacen muchas funciones distintas (programar, clasificar, dialogar) sin cambiar su matriz interna.

V-C. Limitaciones Críticas de los Modelos

A pesar de sus avanzadas capacidades, los LLMs actuales se enfrentan a problemas importantes que se resumen en el Cuadro VI.

Tabla VI
LIMITACIONES PRINCIPALES DE LOS MODELOS DE LENGUAJE

Limitación	¿En qué consiste de forma simple?
Alucinaciones	El modelo inventa respuestas falsas o erróneas, pero las redacta con un tono muy seguro y convincente.
Memoria Limitada	No recuerdan cosas de chats pasados si la información supera el tamaño de su ventana de contexto actual.
Conocimiento Estático	Solo saben lo que había en internet hasta la fecha en que se entrenaron; no se actualizan en tiempo real.
Costos Computacionales	Entrenarlos y usarlos a diario requiere servidores gigantescos, mucha energía y un alto costo económico.

VI. GENERACIÓN AUMENTADA POR RECUPERACIÓN (RAG)

Como solución directa a los problemas de alucinaciones y conocimiento estático de los LLMs, surge la técnica de **Generación Aumentada por Recuperación (RAG)**. Consiste en conectar el modelo con un buscador externo (*retriever*) para extraer datos actualizados o privados justo en el momento de la consulta.

VI-A. Preparación de los Datos: Ingesta y Chunking

Antes de que el sistema pueda buscar información, los documentos deben pasar por un proceso de preparación que se detalla en el Cuadro VII.

Tabla VII
ETAPAS DE INGESTA Y FRAGMENTACIÓN DE DOCUMENTOS

Etapas	¿Qué se hace de forma simple?
Preparación	Se juntan, limpian y centralizan todos los archivos o bases de datos que se van a usar.
Chunking	El texto largo se corta en fragmentos pequeños e independientes llamados <i>chunks</i> (de 200 a 500 tokens).
Vectorización	Cada pedazo se pasa por un modelo de embeddings para convertirlo en un vector con significado semántico.

VI-B. Almacenamiento y Recuperación

VI-B1. Indexación: Los vectores resultantes se guardan en bases de datos vectoriales especializadas (como *Qdrant*, *Pinecone* o *FAISS*). Aquí se almacena tanto el vector (para hacer búsquedas rápidas) como su *metadata*, que guarda el texto original del fragmento.

VI-B2. Consulta o Recuperación: Cuando el usuario hace una pregunta, el sistema busca los fragmentos más relevantes mediante los siguientes pasos:

1. Convierte la pregunta del usuario en un embedding (vector).
2. Calcula la similitud matemática contra todos los vectores guardados en la base de datos.
3. Extrae los mejores resultados (los conocidos como el *top-k* más cercanos).

Esto permite una **"búsqueda por significado"** en lugar de buscar por palabras exactas.

VI-B3. Aumentación y Generación: Los fragmentos recuperados se pegan dentro de una plantilla de texto (*prompt*) junto a la pregunta original del usuario. Este bloque de información sirve como base de conocimientos, obligando al LLM a dar una respuesta basada puramente en la evidencia real encontrada.

VI-C. El Ecosistema Completo de la Arquitectura RAG

En resumen, una arquitectura RAG funciona como un ecosistema dividido en dos fases operativas principales descritas en el Cuadro VIII.

Tabla VIII
FASES PRINCIPALES DE LA ARQUITECTURA RAG

Fase	Acción del Sistema
Pre-producción	Los documentos se limpian, se cortan en <i>chunks</i> , se vectorizan y se guardan en la base de datos.
Producción	El usuario pregunta, el buscador encuentra los mejores datos, se los inyecta al prompt y el LLM responde.

Como se ilustra en la Fig. 4, la combinación de ambas fases permite superar el conocimiento estático del modelo, logrando respuestas mucho más precisas y fundamentadas.

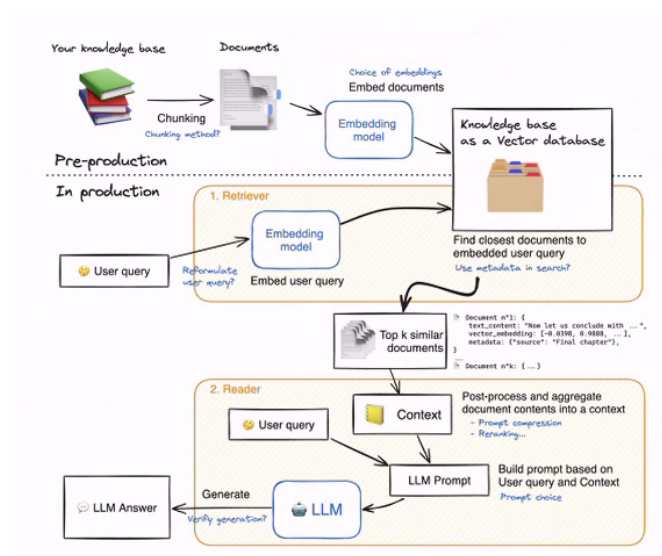


Figura 4. Diagrama general de la Arquitectura RAG y el flujo de los datos.